

Multi-Voltage Crypto Core

The Definitive Tape-out Textbook & Engineering Manual

Lead Author & Architect: Ajay Mallesh

Reviewer: VLSI Signoff Committee : Ajay Mallesh

Technology Node: SAED 32nm Multi-Vt

Scale: ~600,000 instances | Dual Domain (0.95V / 0.75V)

Edition: Version 5.0 (2026)

Scope: Volume 1: Workspace Architecture, RTL Synthesis, Multi-Voltage Theory, Floorplanning, Power Delivery, MCMM, and Detail Placement.

Note: Clock Tree Synthesis (CTS), Global Routing, and Signoff will be addressed in Volume 2.

Multi-Voltage Crypto Core

The Complete Engineering Encyclopedia: RTL to Placement

Lead Author & Architect: Ajay Mallesh

Reviewer: VLSI Signoff Committee : Ajay Mallesh

Technology Node: SAED 32nm Multi-Vt

Design Scale: ~600,000 instances | Dual Voltage Domain (0.95V / 0.75V)

Edition: Version 6.0 (May 2026)

Scope: Volume 1 contains exhaustive theoretical and practical guidelines encompassing Linux Architecture, **Detailed RTL Synthesis Steps & Inputs**, Multi-Voltage Physics, UPF Strategies, Floorplanning, Power Grid Design, MCMM, and Standard Cell Placement.

Preface & Legal Signoff

This document serves as the absolute source of truth for the physical design execution of the Multi-Voltage Crypto Core ASIC. It has been rigorously compiled to bridge the gap between theoretical semiconductor physics and practical, command-line execution using Synopsys IC Compiler II (ICC2) and Design Compiler (DC).

Target Audience: This encyclopedia is written for incoming Junior Physical Design Engineers, Freshers, and Cross-functional Architects who require a deep, foundational understanding of every decision made during the RTL-to-Placement pipeline. It assumes basic digital logic knowledge but builds the physical design concepts from the ground up.

Review & Signoff: By order of the VLSI Signoff Committee, all scripts, math, and methodologies contained within this document have been validated against the SAED32nm rule decks. Deviations from these guidelines without formal committee approval will result in immediate tape-out abort.

Disclaimer: Clock Tree Synthesis (CTS), Auto Routing, Crosstalk Mitigation, and Physical Verification (DRC/LVS) are explicitly outside the scope of this volume and are detailed in Volume 2.

Table of Contents

Chapter 1: Linux Environment & Architecture

- 1.1 Operating System Fundamentals
- 1.2 Directory Hierarchy Construction

Chapter 2: RTL & Logic Synthesis Deep Dive

- 2.1 Register Transfer Level (RTL) Theory
- 2.2 The Inputs to Logic Synthesis (DBs, SDC, UPF, RTL)
- 2.3 The Three-Stage Compilation Pipeline
- 2.4 Synthesis Outputs & Handoff

Chapter 3: Multi-Voltage (MSV) Physics & UPF

- 3.1 Power Dissipation Mathematics
- 3.2 Level Shifters and Isolation Cells
- 3.3 UPF Violations (MV-022)

Chapter 4: ICC2 Initialization & Floorplanning

- 4.1 Physical Database Linking
- 4.2 Core Utilization Mathematics
- 4.3 The Over-Utilization Trap (PLACE-006)

Chapter 5: Power Delivery Network (PG Mesh)

- 5.1 Electromigration & IR Drop
- 5.2 Typo Cascades: Missing Vias (PGR-002)

Chapter 6: MCMM & Parasitic Extraction

6.1 RC Extraction Theory

6.2 Scenario Manager Crash (OPT-041)

Chapter 7: Standard Cell Placement

7.1 Placement Algorithms & Congestion

7.2 DRV Resolution (NDMUI-023 Error)

7.3 Setup Slack (WNS) Analysis & Handoff

Chapter 1: Linux Environment & Architecture

The foundation of any successful semiconductor tape-out begins far before a single line of Verilog is read. It begins with the architecture of the operating system and the project workspace. In VLSI, Windows or graphical environments are entirely obsolete. We operate strictly in high-performance Linux environments using C-Shell (csh) or Bash.

1.1 Why Linux is the Absolute Standard

Synopsys IC Compiler II is a monstrous software suite. Placing and routing 600,000 cells while simultaneously calculating millions of RC delay paths requires massive kernel-level memory management. A typical ICC2 run for a block this size requires a minimum of 128GB of RAM and leverages highly parallelized multi-threading across 32 to 64 CPU cores.

Linux provides the `ulimit` capabilities to uncap memory usage, the raw I/O speeds necessary to write 50GB database files, and the strict file-permission architectures (`chmod`, `chown`) required to prevent junior engineers from accidentally overwriting golden reference files.

1.2 Constructing the Workspace Hierarchy

Database corruption is the leading cause of delayed tape-outs. If an engineer executes a script that overwrites the source RTL or modifies a technology file, the final GDSII sent to the foundry will contain fatal layout-vs-schematic (LVS) mismatches.

To mitigate this, the VLSI Signoff Committee enforces a strict, modular directory hierarchy. Open your Linux terminal and execute the following unified script to build the environment.


```
#!/bin/bash
#
=====
# SCRIPT: workspace_architect.sh
# AUTHOR: Ajay Mallesh
# DESC: Enforces strict directory scaffolding for the Crypto Core
#
=====

# 1. Define the absolute root of the project
export PROJ_ROOT="/home/ajaym/vg_10/crypto_top"
echo "Initializing Workspace at: $PROJ_ROOT"

# Ensure root exists and navigate
mkdir -p $PROJ_ROOT
cd $PROJ_ROOT

# 2. Create isolated data directories
echo "--> Creating structural directories..."
mkdir -p inputs_rtl      # Read-only frontend data
mkdir -p inputs          # Pre-Synthesis DBs & Constraints
mkdir -p scripts         # Execution scripts (TCL)
mkdir -p constraints     # Timing definitions (SDC/MCMM)
mkdir -p CLIBs           # Foundry Technology Databases (NDM)
mkdir -p outputs         # Synthesis Handoffs
mkdir -p reports         # Tool logs and violation reports
mkdir -p work            # ICC2 active database

# 3. Generate execution scripts inside the scripts folder
echo "--> Generating TCL script templates..."
touch scripts/dc_compile.tcl
touch scripts/voltage_area_efficient.tcl
touch scripts/total_cell_area.tcl
```

```
touch scripts/bound_tap_script.tcl
touch scripts/powerplan_crypto_top.tcl
touch scripts/placement_script.tcl
touch scripts/pd_flow.tcl

# 4. Secure the inputs directory (Read-only for users)
chmod 555 inputs_rtl
chmod 555 inputs

# Verify structure
echo "--> Workspace Architecture Complete."
tree .
```

Chapter 2: RTL & Logic Synthesis

Deep Dive

Physical Design cannot exist without Synthesis. Synthesis is the incredibly complex mathematical bridge between abstract human thought (RTL) and physical semiconductor hardware.

2.1 Register Transfer Level (RTL) Theory

Frontend designers write in Verilog or SystemVerilog. They describe behavior, not hardware. They define "Registers" (Flip-Flops) and describe how data "Transfers" between them during a clock cycle.

Consider this basic RTL block for the Crypto Core:

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        cipher_state <= 128'b0;
    else if (enable)
        cipher_state <= data_in ^ round_key; // XOR Logic
end
```

This code tells the simulator to perform an XOR operation. It does **not** specify the use of an SAED32nm XOR2X1_HVT standard cell. It does not

know how fast the cell is, how much power it consumes, or how big it is. It is entirely abstract.

2.2 The Inputs to Logic Synthesis

To convert the abstract RTL into physical gates, Synopsys Design Compiler requires highly specific input files. These are located in our `inputs/` and `inputs_rtl/` directories.

The Required Synthesis Inputs & Their Uses

1. Target Library (The `.db` files):`

If you look in the `inputs/` directory, you will see files like `saed32hvt_ss0p75v125c.db` and `saed32rvt_ss0p95v125c.db`. These are the Synopsys Database files provided by the foundry. They contain the logical function, timing arcs, and power consumption of every standard cell (LEGO brick) the tool is allowed to use.

Usage: Design Compiler reads these to understand exactly how fast an XOR gate is at 0.95V vs 0.75V.

2. The RTL Code (`crypto_top.v`):`

The raw Verilog code describing the cryptographic behavior.

Usage: This is the raw material that gets translated into a logical network.

3. The Initial Constraints (`constraints_mv.sdc`):`

The Synopsys Design Constraints file. It tells the tool the target clock speed (e.g., `create_clock -name clk -period 2.0`).

Usage: The synthesis tool uses this goal to decide whether to map an equation to a small/slow gate or a large/fast gate to meet the 2.0ns deadline.

4. The Power Intent (`crypto_power.upf`):`

The Unified Power Format file defining the 0.95V and 0.75V domains.

Usage: Tells the synthesis tool to strictly insert HVT cells in the 0.75V domain and to insert Level Shifters on the boundaries.

2.3 The Three-Stage Compilation Pipeline

When we execute our `scripts/dc_compile.tcl` script using Synopsys Design Compiler, the tool engages a massive mathematical engine consisting of three phases:

1. Translation (Elaboration)

The tool parses the `crypto_top.v` Verilog syntax and translates it into a technology-independent boolean network known as GTECH (Generic Technology). The ``^`` operator is converted into a generic mathematical XOR node.

2. Logic Optimization

The tool acts as a mathematician. It applies boolean algebra to reduce the equations. If a designer wrote poorly optimized RTL (e.g., $Y = (A \cdot B) + (A \cdot B')$), the tool reduces it to $Y = A$, instantly deleting thousands of unnecessary gates from the design to save area and power.

3. Technology Mapping

This is where physical reality begins. The tool looks at the optimized boolean equations, opens the `.db` library files from the `inputs/` folder, and searches for actual physical cells that match the equations. It swaps the generic XOR node for an actual `XOR2X1_RVT` physical cell that can meet the rules specified in the `.sdc`.

2.4 Synthesis Outputs & The Handoff

Synthesis concludes by writing crucial files into our `outputs/` directory. These files are the direct inputs for ICC2 (Physical Design).

- **crypto_top_netlist_dft.v:** The gate-level netlist. The `_dft` stands for Design-For-Test. Design Compiler has automatically replaced standard Flip-Flops with Scan Flip-Flops (SDFF), wiring them into long shift-register chains so the factory can inject test vectors into the chip after manufacturing.
- **crypto_top_synth.sdc:** The updated timing constraints, ensuring ICC2 understands the exact clock paths and I/O delays.
- **crypto_top_dft.upf:** The updated power intent, now mapped perfectly to the synthesized gate-level hierarchy.
- **crypto_top.scandef:** A structural file detailing the exact ordering of the scan chains. ICC2 will use this later to re-order the chains based on physical placement to save routing congestion.

Chapter 3: Multi-Voltage Physics & UPF

The Crypto Core is an Advanced Low Power (ALP) ASIC. It features two physically and electrically distinct voltage domains: 0.95V and 0.75V. A deep understanding of semiconductor physics is required to manage this architecture.

3.1 Power Dissipation Mathematics

Why do we introduce the massive complexity of a dual-voltage design? The answer lies in the CMOS power equation.

Dynamic vs. Static Power

Dynamic Power (Switching Power):

$$P_{\text{dyn}} = \alpha \cdot C_{\text{eff}} \cdot V_{\text{dd}}^2 \cdot f$$

Where α is the activity factor, C is capacitance, V is supply voltage, and f is clock frequency. Because power scales with the **square** of the voltage, dropping the Key Generation block from 0.95V to 0.75V yields a massive ~40% reduction in dynamic power.

Static Power (Leakage Power):

Even when a transistor is "off", subthreshold current leaks from Source to Drain. To combat this, we use **HVT (High Threshold Voltage)** cells in the 0.75V domain. HVT cells require more voltage to turn on, which severely limits their leakage current, acting as a tight "valve" to save battery life.

3.2 Domain Crossings: Level Shifters

When a signal crosses from the 0.75V PD_LP domain into the 0.95V PD_TOP domain, electrical catastrophe is imminent if untreated.

In a CMOS inverter, a "Logic 1" at the input turns OFF the PMOS and turns ON the NMOS. However, if a 0.75V signal enters a 0.95V inverter, 0.75V is not high enough to fully turn OFF the 0.95V PMOS transistor. Consequently, both the PMOS and NMOS will be partially ON at the same time. This creates a

direct short circuit from VDD to VSS, known as **Crowbar Current**, which will physically burn the chip.

We solve this using **Level Shifters (LS)**. A Level Shifter is a dual-rail amplifier that safely receives 0.75V and outputs a crisp 0.95V signal.

3.3 UPF Violations: The MV-022 Edge Case

The Unified Power Format (UPF) is the TCL-based script that tells the tool how to handle these domains.

Violation Deep Dive: MV-022

Error Log:

```
Warning:  MV-022  Cannot insert level shifter on net
'key_out' crossing from PD_LP to PD_TOP.
```

Root Cause Analysis: This warning appears during ICC2 initialization. It means the tool detected a wire crossing domains, but it doesn't know how to fix it. This happens if you fail to load the physical Level Shifter NDM library in `create_lib`, or if you omit the bridging strategy in your UPF script.

If you ignore this warning and proceed to placement, the tool will wire the 0.75V net directly to the 0.95V pin, destroying the silicon.

The Fix: Ensure your UPF file contains the explicit insertion strategy before execution:

```
set_level_shifter_strategy -domain PD_LP      -rule all      -location s
```

Chapter 4: ICC2 Initialization & Floorplanning

Floorplanning is arguably the most critical stage of Physical Design. A bad floorplan will result in unroutable congestion and impossible timing violations months later in the project schedule.

4.1 Physical Database Linking

We begin by executing `read_verilog` to load the synthesis netlist from the `outputs/` folder, followed immediately by `link_block`. The linking phase acts as a bridge: it matches the text string "AND2X1_HVT" in the Verilog to the physical polygon geometry located in the `CLIBs/` folder.

Edge Case: Unresolved Black Boxes

If `link_block` returns "Unresolved Reference", it means the netlist asks for a cell that does not exist in your loaded NDMs. ICC2 handles this by inserting a hollow "Black Box" in its place. The tool will place and route around this empty box. If you tape out with a black box, your chip will be missing thousands of transistors. Always execute `check_design -checks pre_placement_stage` to verify 100% resolution.

4.2 Core Utilization Mathematics

The outer boundary of the silicon die is determined by **Target Utilization**. Utilization is the ratio of standard cell area to total core area. For a design with 600,000 cells summing to 4.5 million square micrometers (μm^2), targeting an 80% utilization gives:

$$\text{Core Area} = 4,500,000 / 0.80 = 5,625,000 \mu\text{m}^2$$

This leaves 20% of the silicon completely empty, providing essential space for routing wires and clock buffers to be inserted later.

4.3 The Over-Utilization Trap (PLACE-006)

Because we have a 0.75V domain, we must draw a physical "fence" (Voltage Area) on the floorplan. Only logic assigned to PD_LP can be placed inside this fence.

Violation Deep Dive: PLACE-006

Log:

Error: Over utilization of 211.34% for PD_LP. Required area is 2147 μm^2 while available is 1016 μm^2 . (PLACE-006)

The Theory: The raw logical gates assigned to PD_LP only sum to 1016 μm^2 . However, standard cells are not the only things that exist in physical reality. ICC2 must insert 134 massive Level Shifters, hundreds of Well-Tap cells, Tie-High/Low cells, and leave room for CTS buffers. The actual required footprint explodes to 2147 μm^2 .

The Fix: We must manually override the math and pad the area to $\sim 2600 \mu\text{m}^2$ in our script.

4.4 Manufacturing Grid Snapping

You cannot simply draw a box of $2600 \mu\text{m}^2$. The coordinates must snap perfectly to the manufacturing grid, known as **Site Rows**. Standard cells are designed to fit into rows of a specific height. If your voltage area cuts through the middle of a row, the tool cannot legally place cells there.

In SAED32nm, the site height is $1.672 \mu\text{m}$ and the site width is $0.152 \mu\text{m}$.

```

# Script: scripts/voltage_area_efficient.tcl
remove_voltage_area -all

# Target area: 2600. Set initial height to ~40um.
# SNAP TO SITE HEIGHT (1.672)
set h [expr {ceil (40.0 / 1.672) * 1.672 } ] ;# Result: 40.128

# Calculate required width based on snapped height
# SNAP TO SITE WIDTH (0.152)
set w [expr {ceil ((2600.0 / $h) / 0.152) * 0.152 } ] ;# Result:
64.904

set llx 10.016
set lly 10.016
set urx [expr {$llx + $w}]
set ury [expr {$lly + $h}]

create_voltage_area -region [list [list $llx $lly] [list $urx
$ury]] \
    -guard_band {{1.52 1.52}} -power_domains PD_LP

```

Chapter 5: Power Delivery

Network (PG Mesh)

The power grid must deliver huge amounts of current to 600,000 cells simultaneously. If the grid is poorly designed, it falls victim to two fatal physical phenomena: IR Drop and Electromigration.

5.1 Electromigration (EM) and IR Drop Theory

- **IR Drop:** Ohm's Law states $V = I \cdot R$. Metal wires have resistance. As current travels from the external pins to the center of the chip, voltage is lost as heat. If the 0.95V supply drops to 0.85V at the center, the logic gates slow down and fail timing.
- **Electromigration (EM):** If too much current is pushed through a thin wire, the momentum of the electrons will physically knock metal atoms out of place, eventually creating a void (open circuit) that destroys the chip over time.

To prevent both, we build a heavy mesh on the highest, thickest metal layers (Metal 8 and Metal 7). Standard cells sit on Metal 1. We drop "Vias" (vertical pillars) from M8 all the way down to M1 to distribute the power evenly.

5.2 Typo Cascades: The Missing Via (PGR-002)

During the execution of `powerplan_crypto_top.tcl`, the tool drew the upper M8/M7 straps and the lower M1 rails, but failed to connect them vertically.

Violation Deep Dive: PGR-002 Missing Via

Error Log:

```
Failed to parse string via master: VAI12SQ for via definition.
```

```
Error: For option -via_rule, the specified value {via_master: VAI12SQ} is invalid. (PGR-002)
```

Root Cause Analysis: In TCL scripting, syntax is unforgiving. The script misspelled the via as VAI12SQ instead of the correct foundry name VIA12SQ. Because the via didn't exist, the `set_pg_strategy_via_rule` command failed silently.

When `compile_pg` executed, it found no valid rule for connecting M2 to M1. It left a microscopic physical air-gap in the silicon. Running `check_pg_connectivity` resulted in thousands of "Starvation" errors, meaning the cells had absolutely no power.

The Fix: Correct the spelling in the script. If unsure of a tech name, always query the database: `get_via_masters -filter "layer1_name==M1"`.

5.3 The Corrected Powerplan Execution

```

# 1. Logical Net Connections
connect_pg_net -net VSS [get_pins -hierarchical "*/VSS"]
connect_pg_net -net VDD_DEFAULT [get_pins -hierarchical "*/VDD"]
connect_pg_net -net VDD_LP [get_pins -of_objects [get_cells -
hierarchical -filter "power_domain==PD_LP"] -filter
"pin_type==power"]

# 2. Build the Global Mesh (M8 Vertical, M7 Horizontal)
create_pg_mesh_pattern P_mesh_global -layers {
    {{vertical_layer: M8} {width: 2.0} {pitch: 15} {nets:
{VDD_DEFAULT VSS VDD_LP}}}
    {{horizontal_layer: M7} {width: 2.0} {pitch: 15} {nets:
{VDD_DEFAULT VSS VDD_LP}}}
}
set_pg_strategy S_mesh -pattern P_mesh_global -core

# 3. Create Standard Cell Rails (M1)
create_pg_std_cell_conn_pattern P_std_cell_rail
set_pg_strategy S_std_cell_rail_VDD -pattern P_std_cell_rail -nets
{VSS VDD_DEFAULT} -power_domains PD_TOP
set_pg_strategy S_std_cell_rail_VDDL -pattern P_std_cell_rail -
nets {VSS VDD_LP} -power_domains PD_LP

# 4. The Corrected Via Rule
set_pg_strategy_via_rule S_via_rule \
    -via_rule { {{intersection: adjacent} {via_master: VIA12SQ}} }

# 5. Execute Compilation
compile_pg -strategies {S_mesh S_std_cell_rail_VDD
S_std_cell_rail_VDDL} -via_rule S_via_rule

```

Chapter 6: MCMM & Parasitic Extraction

Before placing cells, the ICC2 engine must understand the physical timing constraints of reality. A logical netlist will still fail if wire delays are ignored. This is configured via Multi-Corner Multi-Mode (MCMM).

6.1 Parasitic Extraction Theory (TLU+)

Every wire in a chip acts as a Resistor (R) and a Capacitor (C). The longer the wire, the higher the RC delay, the slower the signal propagation. The foundry characterizes these physics and provides them in .tluplus lookup tables.

Violation Deep Dive: The Scenario Crash (OPT-041)

Error Log:

```
Warning: no valid parasitic for ss_125c corner(LATE)
(NEX-018)
```

```
Error: Scenario Manager creation failed. Optimization
aborted. (OPT-041)
```

Root Cause Analysis: The physical database uses generic names like "M1" for layers. However, the .tluplus physics file uses foundry-specific names like "METAL1". If you load the TLU+ file but forget the **Layer Mapping File**

(.map), the tool cannot link the physical geometries to the resistance tables. The extraction engine goes blind, timing cannot be calculated, and the ICC2 optimization engine crashes immediately.

The Fix: Always declare the -layermap flag during parasitic extraction.

The Correct MCMM Extraction Script

```
# Script: constraints/mcmm_setup.tcl
set MAP "./inputs/tech/saed32nm_tf_itf_tluplus.map"

# Load the parasitics WITH the layermap!
read_parasitic_tech -tlup ./inputs/tech/saed32nm_1p9m_Cmax.tluplus \
    -layermap $MAP -name max_tlup
read_parasitic_tech -tlup ./inputs/tech/saed32nm_1p9m_Cmin.tluplus \
    -layermap $MAP -name min_tlup

# Assign parasitics to the manufacturing corners
set_parasitic_parameters -corner ss_125c -early_spec max_tlup -
late_spec max_tlup
set_parasitic_parameters -corner ff_m40c -early_spec min_tlup -
late_spec min_tlup
```

Chapter 7: Standard Cell Placement & DRVs

This chapter represents the culmination of Volume 1. The Placement stage is where the ICC2 engine determines the precise (X,Y) coordinates of all 600,000 standard cells. The objective is to minimize total wire length while guaranteeing timing constraints are met and preventing any physical overlap.

7.1 Placement Algorithms & Congestion Theory

Executing `place_opt` triggers three distinct mathematical phases:

1. **Global Placement:** The tool ignores physical overlaps. It treats cells like masses attached by springs (wires) and places them globally to minimize total wire length using quadratic equations.
2. **Legalization:** The tool takes the overlapping blobs of cells and mathematically "snaps" them to the nearest standard-cell site rows, ensuring zero overlaps and adherence to grid rules.
3. **Detail Placement:** The tool makes microscopic adjustments, swapping adjacent cells or flipping them horizontally to squeeze out a few extra picoseconds of timing slack.

During placement, the chip is divided into Global Routing Cells (GRCs). If the tool packs too many highly-connected standard cells inside a single GRC, the number of required routing wires will exceed the available physical metal tracks above that cell. If localized congestion exceeds ~2%, the detailed router will fail entirely in the final stages. We monitor this via `report_congestion`.

7.2 Design Rule Violations (DRVs): Max Capacitance

Physics limits how much "electrical weight" a single logic gate can drive. After placement, we ran `report_constraint -all_violators` and found four **max_capacitance** violations occurring strictly on the `scan_ff` test paths.

Violation Deep Dive: The NDMUI-023 Sizing Error

To fix a capacitance violation, an engineer must upsize the driving cell (e.g., swapping a weak X1 drive strength for a stronger X2). Initially, the following command was attempted:

```
size_cell                u_cipher_hp/round_data_reg_42__115_  
[get_lib_cells */DFFARX2_RVT]
```

The Tool Rejection:

```
Error: Cannot size cell... new lib_cell DFFARX2_RVT not  
compatible with SDFFARX1_HVT (NDMUI-023)
```

Root Cause Analysis: Because the input netlist (crypto_top_netlist_dft.v) contains synthesized factory test chains, the existing cell was an **SDFF** (Scan D-Flip-Flop), which contains extra pins for testing (Scan In, Scan Enable). A standard **DFF** lacks these pins. Forcing the swap would leave the factory test wires floating in silicon, permanently breaking the testability of the chip!

The Fix: You must respect the architectural footprint. You must upsize to the equivalent Scan cell:

```
size_cell          u_cipher_hp/round_data_reg_42__115_  
[get_lib_cells */SDFFARX2_HVT]
```

7.3 Setup Slack (WNS) Analysis & Handoff

Following the resolution of all DRVs, we run `report_qor -summary`. Because the clock tree has not been built yet, the clock arrives at every flip-flop in exactly 0.00ns (Ideal Clocking). Therefore, we strictly analyze **Setup Timing (Worst Negative Slack - WNS)** in the worst-case `ss_125c` corner.


```

Startpoint: u_cipher_hp/round_state_reg_45_12/Q
Endpoint: u_cipher_hp/round_state_reg_46_12/D
Path Group: main_clk
Path Type: max

```

Point	Incr	Path

clock main_clk (rise edge)	0.00	0.00
u_cipher_hp/round_state_reg_45_12/CLK (DFFARX1)	0.00	0.00 r
u_cipher_hp/round_state_reg_45_12/Q (DFFARX1)	0.12	0.12 f
u_cipher_hp/U892/Y (XOR2X1_RVT)	0.22	0.22 r
... [Logic Delays] ...		
u_cipher_hp/round_state_reg_46_12/D	0.80	0.80 r
data arrival time		0.80
clock main_clk (rise edge)	2.00	2.00
clock uncertainty	-0.15	1.85
library setup time	-0.03	1.82
data required time		1.82

data required time		1.82
data arrival time		-0.80

slack (MET)		1.02

Engineering Analysis: Our target clock period is 2.00ns (500MHz). After accounting for realistic clock jitter/uncertainty (-0.15ns) and the internal setup time of the flip-flop (-0.03ns), the data signal *must* arrive by 1.82ns. Because our placement optimization was highly efficient, the data actually arrives in just 0.80ns. We have a massive, healthy positive setup slack of **+1.02ns**.

End of Volume 1: Placement Hand-off

The standard cells are legally placed, the 0.95V / 0.75V power grids are robust and short-free, all Design Rule Violations (DRVs) are resolved, and the data paths have immense positive setup timing slack. The physical database is mathematically proven and structurally sound.

The design is now ready for the next engineering phases: building the synchronized clock network (CTS) and routing the final metal data paths. These stages will be covered in **Volume 2: Clock Tree Synthesis, Detail Routing, and Physical Signoff**.